Paballo Pholo

Graduate Assessment 1 - 03/12/2025

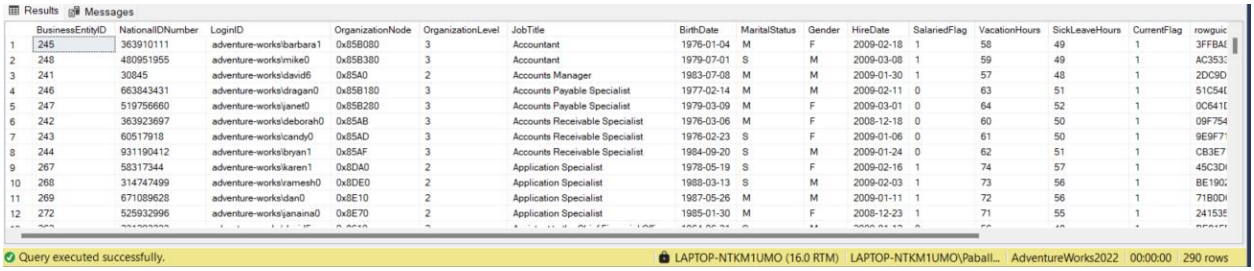
USE [AdventureWorks2022];

1. Write a query in SQL to retrieve all rows and columns from the employee table in the Adventureworks database. Sort the result set in ascending order on jobtitle

SELECT *

FROM [HumanResources].[Employee]

ORDER BY [JobTitle] ASC;



Results Messages

	BusinessEntityID	NationalIDNumber	LoginID	OrganizationNode	OrganizationLevel	JobTitle	BirthDate	MaritalStatus	Gender	HireDate	SalariedFlag	VacationHours	SickLeaveHours	CurrentFlag	rowguic
1	245	363910111	adventure-works\barbara1	0x85B080	3	Accountant	1976-01-04	M	F	2009-02-18	1	58	49	1	3FFBAI
2	248	480951955	adventure-works\mike0	0x85B380	3	Accountant	1979-07-01	S	M	2009-03-08	1	59	49	1	AC353
3	241	30845	adventure-works\david6	0x85A0	2	Accounts Manager	1983-07-08	M	M	2009-01-30	1	57	48	1	2DC9D
4	246	663843431	adventure-works\dragan0	0x85B180	3	Accounts Payable Specialist	1977-02-14	M	M	2009-02-11	0	63	51	1	51C54I
5	247	519756660	adventure-works\janet0	0x85B280	3	Accounts Payable Specialist	1979-03-09	M	F	2009-03-01	0	64	52	1	0C641I
6	242	363923697	adventure-works\deborah0	0x85AB	3	Accounts Receivable Specialist	1976-03-06	M	F	2008-12-18	0	60	50	1	09F754
7	243	60517918	adventure-works\candy0	0x85AD	3	Accounts Receivable Specialist	1976-02-23	S	F	2009-01-06	0	61	50	1	9E9F7
8	244	931190412	adventure-works\bryan1	0x85AF	3	Accounts Receivable Specialist	1984-09-20	S	M	2009-01-24	0	62	51	1	CB3E7
9	267	58317344	adventure-works\karen1	0x8DA0	2	Application Specialist	1978-05-19	S	F	2009-02-16	1	74	57	1	45C3D
10	268	314747499	adventure-works\ramesh0	0x8DE0	2	Application Specialist	1988-03-13	S	M	2009-02-03	1	73	56	1	BE190
11	269	671089628	adventure-works\dan0	0x8E10	2	Application Specialist	1987-05-26	M	M	2009-01-11	1	72	56	1	71B0D
12	272	525932996	adventure-works\janaina0	0x8E70	2	Application Specialist	1985-01-30	M	F	2008-12-23	1	71	55	1	24153

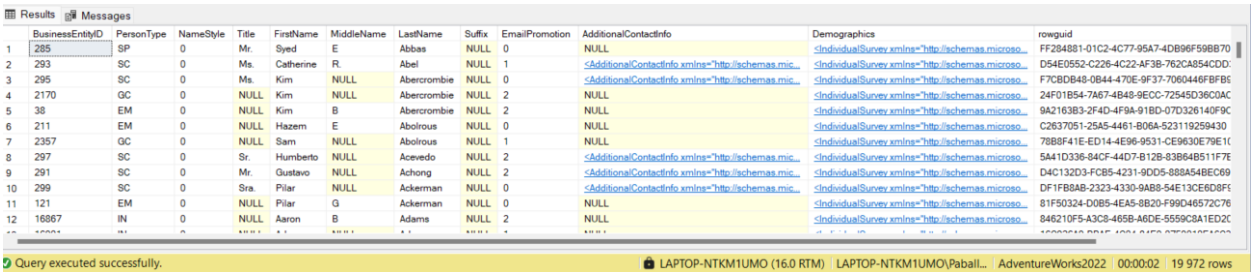
Query executed successfully. LAPTOP-NTKM1UMO (16.0 RTM) LAPTOP-NTKM1UMO\Paball... AdventureWorks2022 00:00:00 290 rows

2. Write a query in SQL to retrieve all rows and columns from the employee table using table aliasing in the Adventureworks database. Sort the output in ascending order on lastname.

SELECT *

FROM [Person].[Person] AS PersonDetails

ORDER BY LastName ASC;



	BusinessEntityID	PersonType	NameStyle	Title	FirstName	MiddleName	LastName	Suffix	EmailPromotion	AdditionalContactInfo	Demographics	rowguid
1	285	SP	0	Mr.	Syed	E	Abbas	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	FF284881-01C2-4C77-95A7-4DB96F59BB70
2	293	SC	0	Ms.	Catherine	R.	Abel	NULL	1	<AdditionalContactInfo xmlns="http://schemas.mic...	<IndividualSurvey xmlns="http://schemas.microso...	D54E0552-C226-4C22-AF3B-762CA854CDD3
3	295	SC	0	Ms.	Kim	NULL	Abercrombie	NULL	0	<AdditionalContactInfo xmlns="http://schemas.mic...	<IndividualSurvey xmlns="http://schemas.microso...	F7CBDB48-0B44-470E-9F37-7060446FBFB5
4	2170	GC	0	NULL	Kim	NULL	Abercrombie	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	24F01B54-7A67-4B48-9ECC-72545D36C0AC
5	38	EM	0	NULL	Kim	B	Abercrombie	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	9A2163B3-2F4D-4F9A-91BD-07D326140F9C
6	211	EM	0	NULL	Hazem	E	Abolrous	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	C2637051-25A5-4461-B06A-523119259430
7	2357	GC	0	NULL	Sam	NULL	Abolrous	NULL	1	NULL	<IndividualSurvey xmlns="http://schemas.microso...	78B8F41E-ED14-4E96-9531-CE9630E79E1C
8	297	SC	0	Sr.	Humberto	NULL	Acevedo	NULL	2	<AdditionalContactInfo xmlns="http://schemas.mic...	<IndividualSurvey xmlns="http://schemas.microso...	5A41D336-84CF-44D7-B12B-83B64B511F7E
9	291	SC	0	Mr.	Gustavo	NULL	Achong	NULL	2	<AdditionalContactInfo xmlns="http://schemas.mic...	<IndividualSurvey xmlns="http://schemas.microso...	D4C132D3-FCB5-4231-9DD5-888A54BEC69
10	299	SC	0	Sra.	Pilar	NULL	Ackerman	NULL	0	<AdditionalContactInfo xmlns="http://schemas.mic...	<IndividualSurvey xmlns="http://schemas.microso...	DF1FB8AB-2323-4330-9AB8-54E13CE6D8F5
11	121	EM	0	NULL	Pilar	G	Ackerman	NULL	0	NULL	<IndividualSurvey xmlns="http://schemas.microso...	81F50324-D0B5-4EA5-8B20-F99D46572C76
12	16867	IN	0	NULL	Aaron	B	Adams	NULL	2	NULL	<IndividualSurvey xmlns="http://schemas.microso...	846210F5-A3C8-465B-A6DE-5559C8A1ED2C

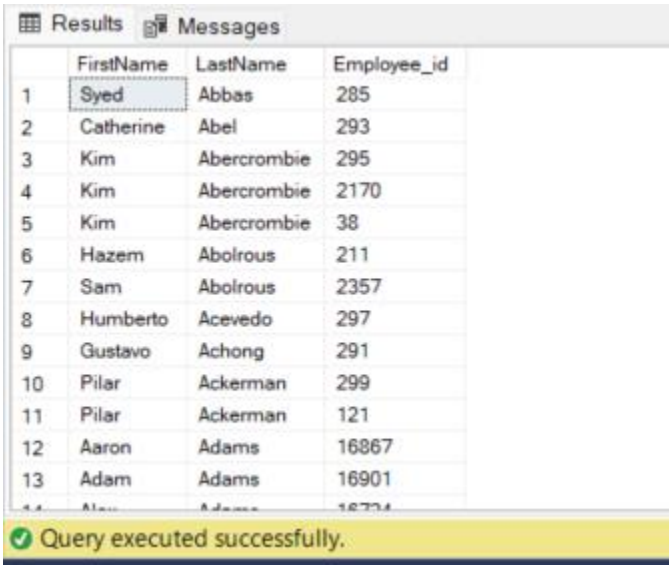
Query executed successfully. LAPTOP-NTKM1UMO (16.0 RTM) LAPTOP-NTKM1UMO\Paball... AdventureWorks2022 00:00:02 19 972 rows

3. Write a query in SQL to return all rows and a subset of the columns (FirstName, LastName, businessentityid) from the person table in the AdventureWorks database. The third column heading is renamed to Employee_id. Arranged the output in ascending order by lastname.

```
SELECT FirstName, LastName, Businessentityid AS Employee_id

FROM [Person].[Person]

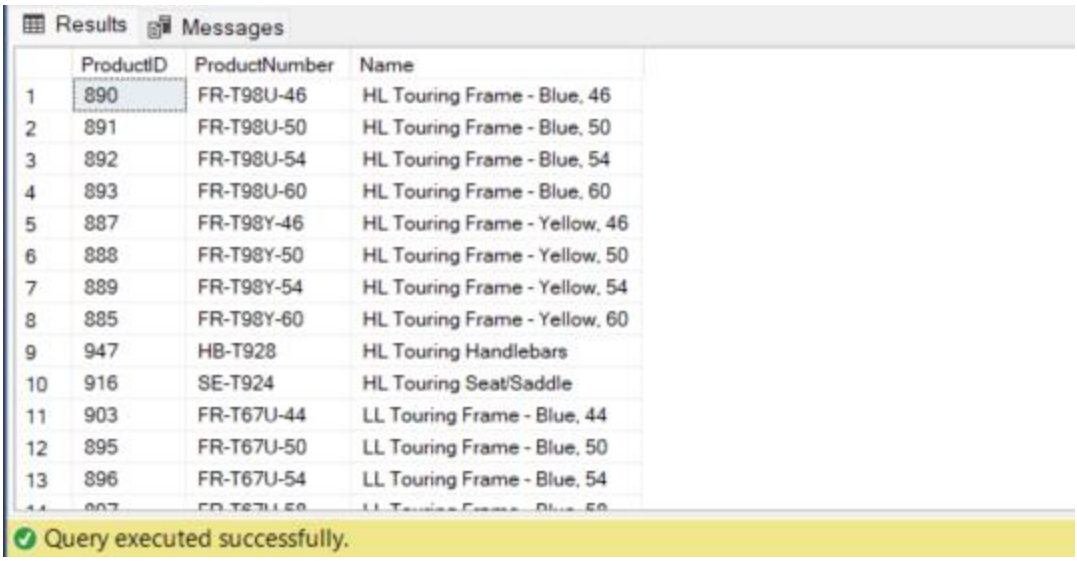
ORDER BY LastName ASC;
```



	FirstName	LastName	Employee_id
1	Syed	Abbas	285
2	Catherine	Abel	293
3	Kim	Abercrombie	295
4	Kim	Abercrombie	2170
5	Kim	Abercrombie	38
6	Hazem	Abolrous	211
7	Sam	Abolrous	2357
8	Humberto	Acevedo	297
9	Gustavo	Achong	291
10	Pilar	Ackerman	299
11	Pilar	Ackerman	121
12	Aaron	Adams	16867
13	Adam	Adams	16901

Query executed successfully.

4. Write a query in SQL to return only the rows for product that have a sellstartdate that is not NULL and a productline of 'T'. Return productid, productnumber, and name. Arranged the output in ascending order on name.



Results Messages

	ProductID	ProductNumber	Name
1	890	FR-T98U-46	HL Touring Frame - Blue, 46
2	891	FR-T98U-50	HL Touring Frame - Blue, 50
3	892	FR-T98U-54	HL Touring Frame - Blue, 54
4	893	FR-T98U-60	HL Touring Frame - Blue, 60
5	887	FR-T98Y-46	HL Touring Frame - Yellow, 46
6	888	FR-T98Y-50	HL Touring Frame - Yellow, 50
7	889	FR-T98Y-54	HL Touring Frame - Yellow, 54
8	885	FR-T98Y-60	HL Touring Frame - Yellow, 60
9	947	HB-T928	HL Touring Handlebars
10	916	SE-T924	HL Touring Seat/Saddle
11	903	FR-T67U-44	LL Touring Frame - Blue, 44
12	895	FR-T67U-50	LL Touring Frame - Blue, 50
13	896	FR-T67U-54	LL Touring Frame - Blue, 54

Query executed successfully.

```
SELECT ProductID, ProductNumber, Name

FROM [Production].[Product]

WHERE [SellStartDate] IS NOT NULL AND [ProductLine] = 'T'

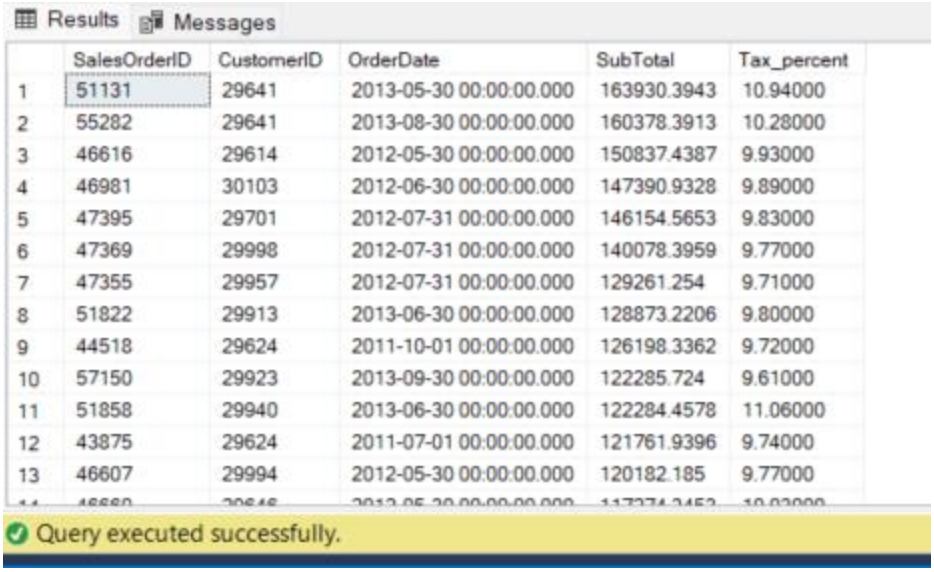
ORDER BY Name ASC;
```

5. Write a query in SQL to return all rows from the salesorderheader table in Adventureworks database and calculate the percentage of tax on the subtotal have decided. Return salesorderid, customerid, orderdate, subtotal, percentage of tax column. Arranged the result set in ascending order on subtotal.

```
SELECT SalesOrderID, CustomerID, OrderDate, SubTotal, ((TaxAmt/SubTotal)*100.0) AS Tax_percent

FROM [Sales].[SalesOrderHeader]

ORDER BY SubTotal DESC;
```



Results Messages

	SalesOrderID	CustomerID	OrderDate	SubTotal	Tax_percent
1	51131	29641	2013-05-30 00:00:00.000	163930.3943	10.94000
2	55282	29641	2013-08-30 00:00:00.000	160378.3913	10.28000
3	46616	29614	2012-05-30 00:00:00.000	150837.4387	9.93000
4	46981	30103	2012-06-30 00:00:00.000	147390.9328	9.89000
5	47395	29701	2012-07-31 00:00:00.000	146154.5653	9.83000
6	47369	29998	2012-07-31 00:00:00.000	140078.3959	9.77000
7	47355	29957	2012-07-31 00:00:00.000	129261.254	9.71000
8	51822	29913	2013-06-30 00:00:00.000	128873.2206	9.80000
9	44518	29624	2011-10-01 00:00:00.000	126198.3362	9.72000
10	57150	29923	2013-09-30 00:00:00.000	122285.724	9.61000
11	51858	29940	2013-06-30 00:00:00.000	122284.4578	11.06000
12	43875	29624	2011-07-01 00:00:00.000	121761.9396	9.74000
13	46607	29994	2012-05-30 00:00:00.000	120182.185	9.77000

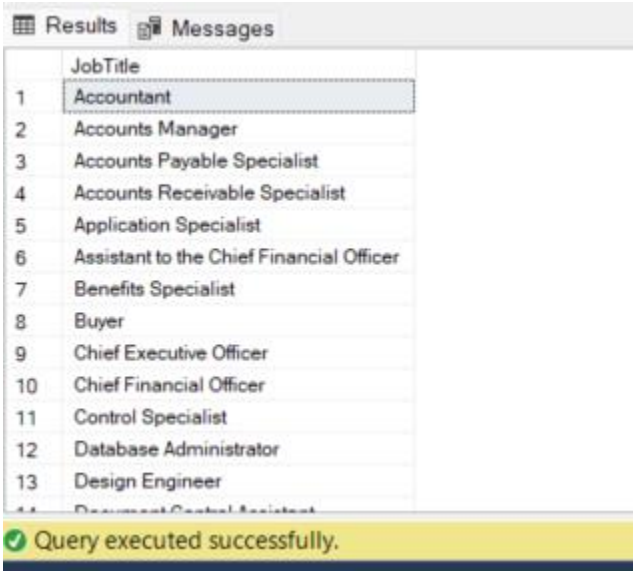
Query executed successfully.

6. Write a query in SQL to create a list of unique jobtitles in the employee table in Adventureworks database. Return jobtitle column and arranged the resultset in ascending order.

```
SELECT DISTINCT [JobTitle]

FROM [HumanResources].[Employee]

ORDER BY [JobTitle] ASC;
```



Results Messages

	JobTitle
1	Accountant
2	Accounts Manager
3	Accounts Payable Specialist
4	Accounts Receivable Specialist
5	Application Specialist
6	Assistant to the Chief Financial Officer
7	Benefits Specialist
8	Buyer
9	Chief Executive Officer
10	Chief Financial Officer
11	Control Specialist
12	Database Administrator
13	Design Engineer

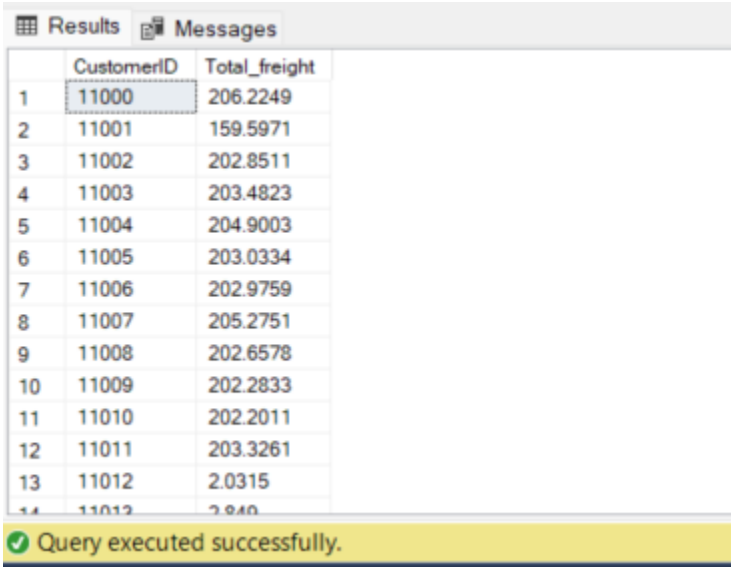
Query executed successfully.

7. Write a query in SQL to calculate the total freight paid by each customer. Return customerid and total freight. Sort the output in ascending order on customerid.

```
SELECT [CustomerID], SUM([Freight]) AS Total_freight

FROM [Sales].[SalesOrderHeader]

ORDER BY [CustomerID] ASC;
```



Results | Messages

	CustomerID	Total_freight
1	11000	206.2249
2	11001	159.5971
3	11002	202.8511
4	11003	203.4823
5	11004	204.9003
6	11005	203.0334
7	11006	202.9759
8	11007	205.2751
9	11008	202.6578
10	11009	202.2833
11	11010	202.2011
12	11011	203.3261
13	11012	2.0315
14	11013	2.849

Query executed successfully.

8. Write a query in SQL to find the average and the sum of the subtotal for every customer. Return customerid, average and sum of the subtotal. Grouped the result on customerid and salespersonid. Sort the result on customerid column in descending order

```
SELECT CustomerID, SalesPersonID, AVG(SubTotal) AS avg_subtotal, SUM(SubTotal) AS sum_subtotal

FROM [Sales].[SalesOrderHeader]

GROUP BY [CustomerID], [SalesPersonID]

ORDER BY CustomerID DESC;
```

Results Messages

	CustomerID	SalesPersonID	avg_subtotal	sum_subtotal
1	30118	275	34638.1521	207828.9131
2	30118	277	35369.8284	70739.6569
3	30117	275	77171.7928	463030.757
4	30117	277	58954.1365	353724.8193
5	30116	276	46778.5502	187114.2011
6	30115	289	1114.6949	8917.5594
7	30114	290	1456.6238	11652.9911
8	30113	282	34148.2363	273185.8908
9	30112	280	93591.6217	93591.6217
10	30112	284	54909.3785	384365.6498
11	30111	277	69131.6007	276526.4031
12	30110	276	406.3188	1625.2755
13	30109	275	59857.2605	119714.521

Query executed successfully.

9. Write a query in SQL to retrieve total quantity of each productid which are in shelf of 'A' or 'C' or 'H'. Filter the results for sum quantity is more than 500. Return productid and sum of the quantity. Sort the results according to the productid in ascending order

```
SELECT [ProductID],SUM([Quantity]) AS total_quantity

FROM [Production].[ProductInventory]

WHERE [Shelf] LIKE '[ACH]'

GROUP BY [ProductID]

HAVING SUM([Quantity]) > 500

ORDER BY ProductID ASC;
```

Results Messages

	ProductID	total_quantity
1	1	761
2	2	791
3	3	909
4	4	900
5	316	532
6	317	593
7	319	797
8	320	1136
9	321	1750
10	322	1684
11	323	1684
12	324	1629
13	325	1210

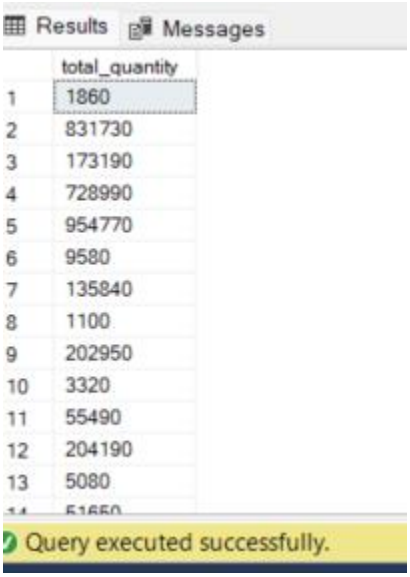
Query executed successfully.

10. Write a query in SQL to find the total quentity for a group of locationid multiplied by 10

```
SELECT SUM([Quantity]) * 10  AS total_quantity

FROM [Production].[ProductInventory]

GROUP BY [LocationID];
```


Results Messages

	total_quantity
1	1860
2	831730
3	173190
4	728990
5	954770
6	9580
7	135840
8	1100
9	202950
10	3320
11	55490
12	204190
13	5080

Query executed successfully.

11. Write a query in SQL to find the persons whose last name starts with letter 'L'. Return BusinessEntityID, FirstName, LastName, and PhoneNumber. Sort the result on lastname and firstname.

```
SELECT person.[BusinessEntityID],[FirstName],[LastName],phone.[PhoneNumber] AS Person_Phone

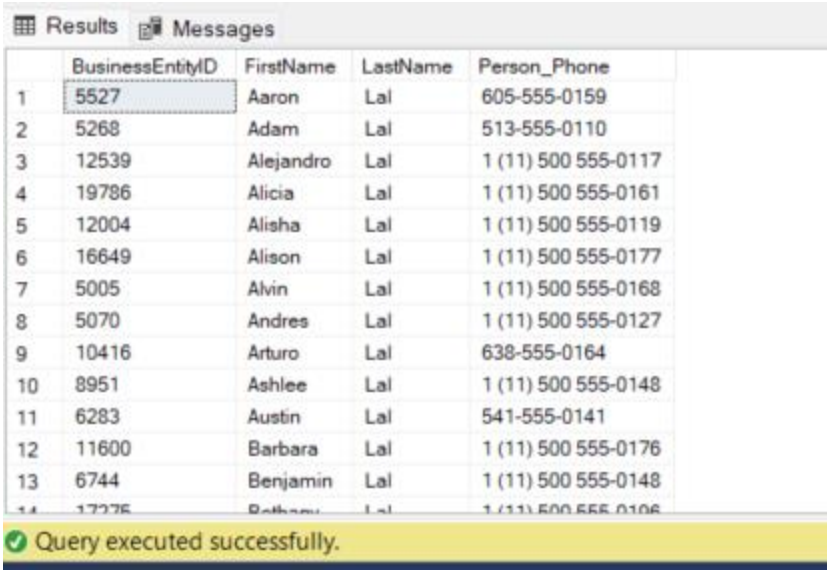
FROM [Person].[Person] as person

JOIN [Person].[PersonPhone] AS phone

ON person.BusinessEntityID = phone.BusinessEntityID

WHERE [LastName] LIKE 'L%'

ORDER BY [LastName], [FirstName] ASC;
```



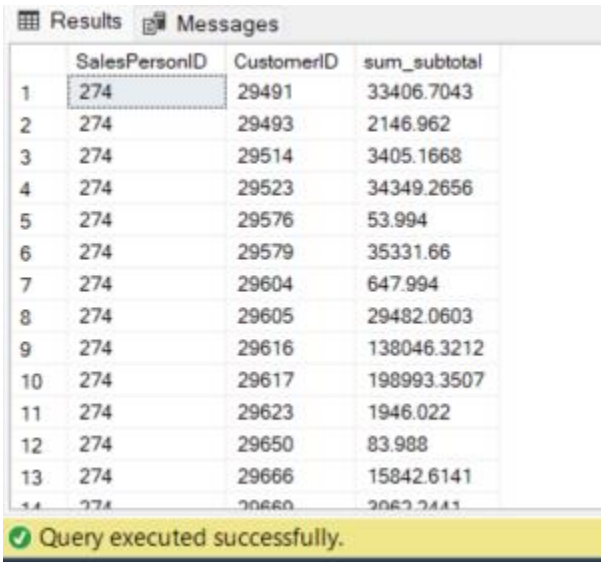
Results Messages

	BusinessEntityID	FirstName	LastName	Person_Phone
1	5527	Aaron	Lal	605-555-0159
2	5268	Adam	Lal	513-555-0110
3	12539	Alejandro	Lal	1 (11) 500 555-0117
4	19786	Alicia	Lal	1 (11) 500 555-0161
5	12004	Alisha	Lal	1 (11) 500 555-0119
6	16649	Alison	Lal	1 (11) 500 555-0177
7	5005	Alvin	Lal	1 (11) 500 555-0168
8	5070	Andres	Lal	1 (11) 500 555-0127
9	10416	Arturo	Lal	638-555-0164
10	8951	Ashlee	Lal	1 (11) 500 555-0148
11	6283	Austin	Lal	541-555-0141
12	11600	Barbara	Lal	1 (11) 500 555-0176
13	6744	Benjamin	Lal	1 (11) 500 555-0148

Query executed successfully.

12. Write a query in SQL to find the sum of subtotal column. Group the sum on distinct salespersonid and customerid. Rolls up the results into subtotal and running total. Return salespersonid, customerid and sum of subtotal column i.e. sum_subtotal.

```
SELECT DISTINCT SalesPersonID, [CustomerID], SUM(SubTotal) AS sum_subtotal
FROM [Sales].[SalesOrderHeader]
WHERE [SalesPersonID] IS NOT NULL
GROUP BY [SalesPersonID], [CustomerID]
ORDER BY [SalesPersonID], [CustomerID] ASC;
```



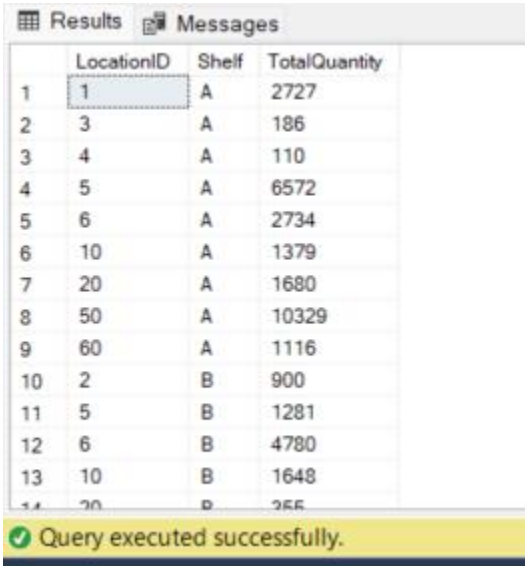
Results Messages

	SalesPersonID	CustomerID	sum_subtotal
1	274	29491	33406.7043
2	274	29493	2146.962
3	274	29514	3405.1668
4	274	29523	34349.2656
5	274	29576	53.994
6	274	29579	35331.66
7	274	29604	647.994
8	274	29605	29482.0603
9	274	29616	138046.3212
10	274	29617	198993.3507
11	274	29623	1946.022
12	274	29650	83.988
13	274	29666	15842.6141

Query executed successfully.

13. Write a query in SQL to find the sum of the quantity of all combination of group of distinct locationid and shelf column. Return locationid, shelf and sum of quantity as TotalQuantity.

```
SELECT DISTINCT [LocationID], [Shelf], SUM([Quantity]) AS TotalQuantity
FROM [Production].[ProductInventory]
GROUP BY [LocationID], [Shelf];
```



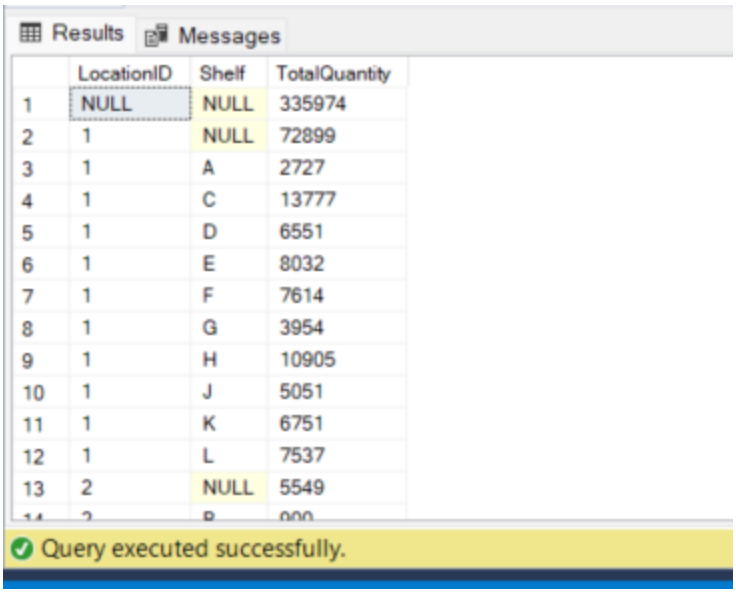
Results Messages

	LocationID	Shelf	TotalQuantity
1	1	A	2727
2	3	A	186
3	4	A	110
4	5	A	6572
5	6	A	2734
6	10	A	1379
7	20	A	1680
8	50	A	10329
9	60	A	1116
10	2	B	900
11	5	B	1281
12	6	B	4780
13	10	B	1648
14	20	B	366

Query executed successfully.

14. Write a query in SQL to find the sum of the quantity with subtotal for each locationid. Group the results for all combination of distinct locationid and shelf column. Rolls up the results into subtotal and running total. Return locationid, shelf and sum of quantity as TotalQuantity.

```
SELECT DISTINCT [LocationID], [Shelf], SUM([Quantity]) AS TotalQuantity
FROM [Production].[ProductInventory]
GROUP BY ROLLUP([LocationID], [Shelf]);
```



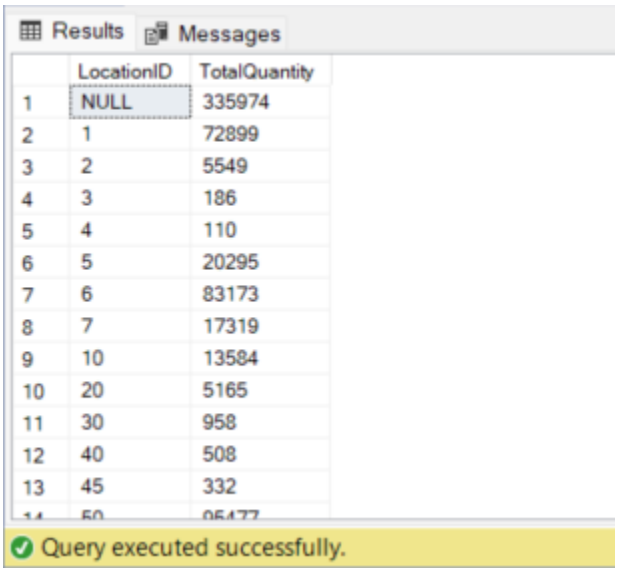
Results Messages

	LocationID	Shelf	TotalQuantity
1	NULL	NULL	335974
2	1	NULL	72899
3	1	A	2727
4	1	C	13777
5	1	D	6551
6	1	E	8032
7	1	F	7614
8	1	G	3954
9	1	H	10905
10	1	J	5051
11	1	K	6751
12	1	L	7537
13	2	NULL	5549

Query executed successfully.

15. Write a query in SQL to find the total quantity for each locationid and calculate the grand total for all locations. Return locationid and total quantity. Group the results on locationid.

```
SELECT  [LocationID], SUM([Quantity]) AS TotalQuantity
FROM [Production].[ProductInventory]
GROUP BY ROLLUP([LocationID])
ORDER BY [LocationID] ASC;
```



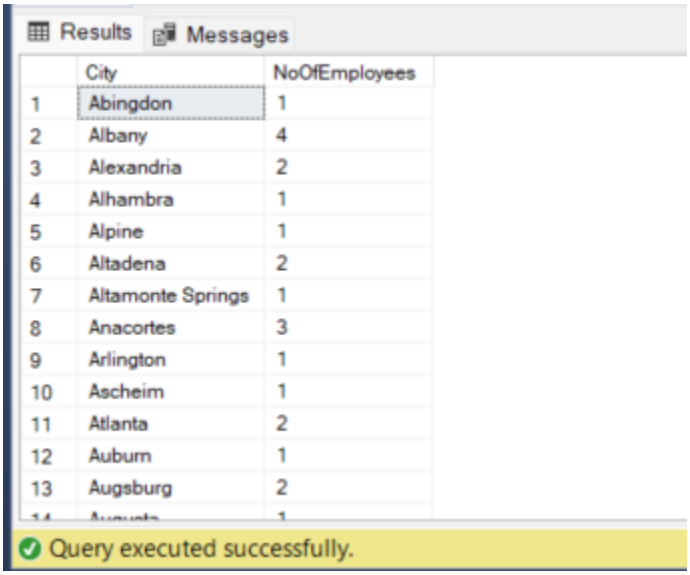
Results Messages

	LocationID	TotalQuantity
1	NULL	335974
2	1	72899
3	2	5549
4	3	186
5	4	110
6	5	20295
7	6	83173
8	7	17319
9	10	13584
10	20	5165
11	30	958
12	40	508
13	45	332

Query executed successfully.

16. Write a query in SQL to retrieve the number of employees for each City. Return city and number of employees. Sort the result in ascending order on city

```
SELECT a.[City], COUNT(b.[BusinessEntityID]) AS NoOfEmployees
FROM [Person].[BusinessEntityAddress] AS b
JOIN [Person].[Address] AS a
ON b.[AddressID] = a.[AddressID]
GROUP BY a.[City]
ORDER BY a.[City] ASC;
```



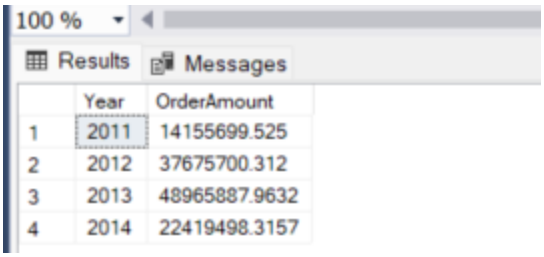
Results | Messages

	City	NoOfEmployees
1	Abingdon	1
2	Albany	4
3	Alexandria	2
4	Alhambra	1
5	Alpine	1
6	Altadena	2
7	Altamonte Springs	1
8	Anacortes	3
9	Arlington	1
10	Ascheim	1
11	Atlanta	2
12	Auburn	1
13	Augsburg	2

Query executed successfully.

17. Write a query in SQL to retrieve the total sales for each year. Return the year part of order date and total due amount. Sort the result in ascending order on year part of order date.

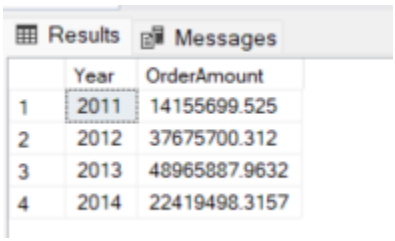
```
SELECT YEAR([OrderDate]) AS Year, SUM([TotalDue]) AS OrderAmount
FROM [Sales].[SalesOrderHeader]
GROUP BY YEAR([OrderDate])
ORDER BY YEAR([OrderDate]) ASC;
```



	Year	OrderAmount
1	2011	14155699.525
2	2012	37675700.312
3	2013	48965887.9632
4	2014	22419498.3157

18. Write a query in SQL to retrieve the total sales for each year. Filter the result set for those orders where order year is on or before 2016. Return the year part of orderdate and total due amount. Sort the result in ascending order on year part of order date.

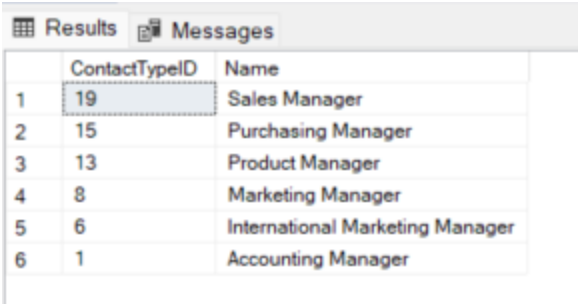
```
SELECT YEAR([OrderDate]) AS Year, SUM([TotalDue]) AS OrderAmount
FROM [Sales].[SalesOrderHeader]
WHERE YEAR([OrderDate]) <= '2026'
GROUP BY YEAR([OrderDate])
ORDER BY YEAR([OrderDate]) ASC;
```



	Year	OrderAmount
1	2011	14155699.525
2	2012	37675700.312
3	2013	48965887.9632
4	2014	22419498.3157

19. Write a query in SQL to find the contacts who are designated as a manager in various departments. Returns ContactTypeID, name. Sort the result set in descending order.

```
SELECT [ContactTypeID],[Name]
FROM [Person].[ContactType]
WHERE [Name] LIKE '%Manager'
ORDER BY [ContactTypeID] DESC;
```



Results Messages

	ContactTypeID	Name
1	19	Sales Manager
2	15	Purchasing Manager
3	13	Product Manager
4	8	Marketing Manager
5	6	International Marketing Manager
6	1	Accounting Manager

20. From the following tables write a query in SQL to make a list of contacts who are designated as 'Purchasing Manager'. Return BusinessEntityID, LastName, and FirstName columns. Sort the result set in ascending order of LastName, and FirstName.

```
SELECT [BusinessEntityID],[LastName],[FirstName]

FROM [Person].[Person] AS p

JOIN [Person].[ContactType] AS c

ON p.[BusinessEntityID] = c.[ContactTypeID]

WHERE c.[Name] = 'Purchasing Manager'

--WRONG
```

21. Write a query in SQL to retrieve the salesperson for each PostalCode who belongs to a territory and SalesYTD is not zero. Return row numbers of each group of PostalCode, last name, salesytd, postalcode column. Sort the salesytd of each postalcode group in descending order. Shorts the postalcode in ascending order.

```
SELECT

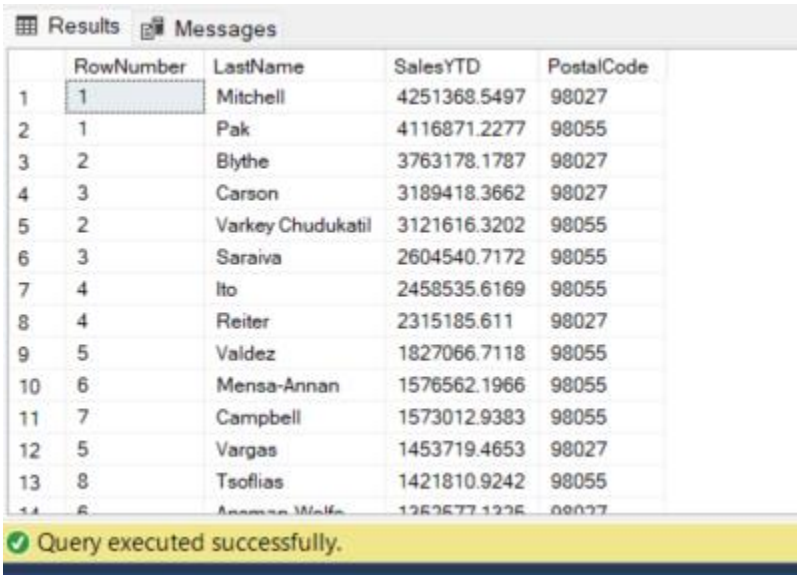
  ROW_NUMBER() OVER(PARTITION BY a.[PostalCode] ORDER BY s.[SalesYTD] DESC) AS RowNumber,

  p.[LastName],

  s.[SalesYTD],

  a.[PostalCode]
```

```
FROM [Sales].[SalesPerson] AS s
JOIN[Person].[Person] AS p
ON s.[BusinessEntityID] = p.[BusinessEntityID]
JOIN [Person].[BusinessEntityAddress] AS b
ON b.[AddressID] = p.[BusinessEntityID]
JOIN [Person].[Address] AS a
ON a.[AddressID] = b.[AddressID]
ORDER BY s.[SalesYTD] DESC, a.[PostalCode] ASC
```



Results Messages

	RowNumber	LastName	SalesYTD	PostalCode
1	1	Mitchell	4251368.5497	98027
2	1	Pak	4116871.2277	98055
3	2	Blythe	3763178.1787	98027
4	3	Carson	3189418.3662	98027
5	2	Varkey Chudukatil	3121616.3202	98055
6	3	Saraiva	2604540.7172	98055
7	4	Ito	2458535.6169	98055
8	4	Reiter	2315185.611	98027
9	5	Valdez	1827066.7118	98055
10	6	Mensa-Annan	1576562.1966	98055
11	7	Campbell	1573012.9383	98055
12	5	Vargas	1453719.4653	98027
13	8	Tsoflias	1421810.9242	98055

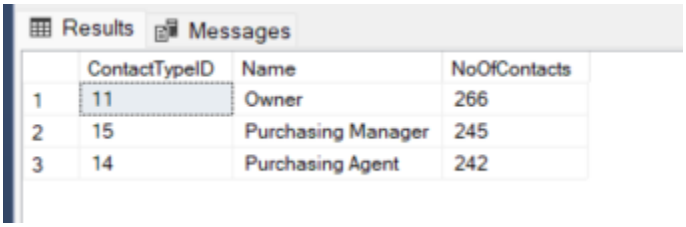
Query executed successfully.

22. Write a query in SQL to count the number of contacts for combination of each type and name. Filter the output for those who have 100 or more contacts. Return ContactTypeID and ContactTypeName and BusinessEntityContact. Sort the result set in descending order on number of contacts.

```
SELECT b.[ContactTypeID], c.[Name], COUNT(*) AS NoOfContacts
FROM [Person].[BusinessEntityContact] AS b
JOIN [Person].[ContactType] AS c
ON b.[ContactTypeID] = c.[ContactTypeID]
GROUP BY b.[ContactTypeID], c.[Name]
```

HAVING COUNT(*) >= 100

ORDER BY NoOfContacts DESC;



Results | Messages

	ContactTypeID	Name	NoOfContacts
1	11	Owner	266
2	15	Purchasing Manager	245
3	14	Purchasing Agent	242

23. Write a query in SQL to retrieve the RateChangeDate, full name (first name, middle name and last name) and weekly salary 40 hours in a week) of employees. In the output the RateChangeDate should appears in date format. Sort the output in ascending order on NameInFull.

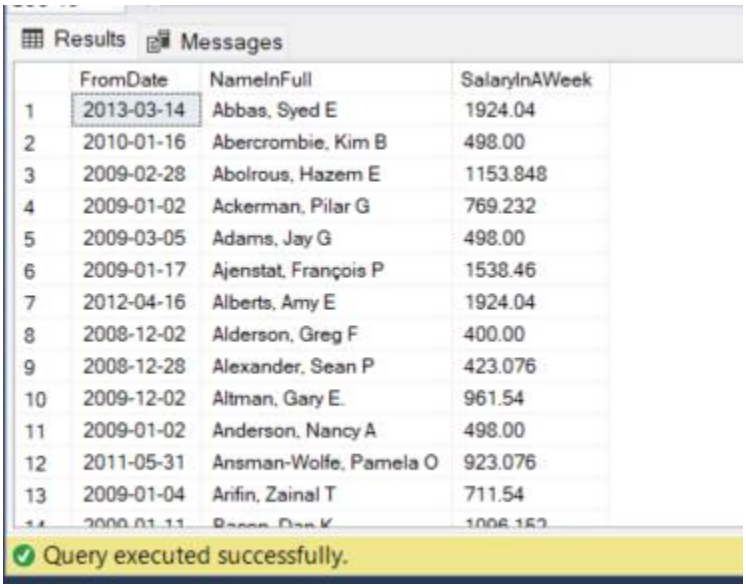
SELECT CONVERT(DATE,[RateChangeDate]) AS FromDate, CONCAT(LastName,', ',FirstName, ' ', COALESCE(MiddleName,' ')) AS NameInFull, [Rate] * 40 AS SalaryInAWeek

FROM [Person].[Person] AS p

JOIN [HumanResources].[EmployeePayHistory] AS e

ON p.BusinessEntityID = e.BusinessEntityID

ORDER BY NameInFull ASC;



Results | Messages

	FromDate	NameInFull	SalaryInAWeek
1	2013-03-14	Abbas, Syed E	1924.04
2	2010-01-16	Abercrombie, Kim B	498.00
3	2009-02-28	Abolrous, Hazem E	1153.848
4	2009-01-02	Ackerman, Pilar G	769.232
5	2009-03-05	Adams, Jay G	498.00
6	2009-01-17	Ajenstat, François P	1538.46
7	2012-04-16	Alberts, Amy E	1924.04
8	2008-12-02	Alderson, Greg F	400.00
9	2008-12-28	Alexander, Sean P	423.076
10	2009-12-02	Altman, Gary E.	961.54
11	2009-01-02	Anderson, Nancy A	498.00
12	2011-05-31	Ansman-Wolfe, Pamela O	923.076
13	2009-01-04	Arifin, Zainal T	711.54
14	2009-01-11	Bacon, Dan K	1096.152

Query executed successfully.

24. Write a query in SQL to calculate and display the latest weekly salary of each employee. Return RateChangeDate, full name (first name, middle name and last

name) and weekly salary 40 hours in a week) of employees Sort the output in ascending order on NameInFull.

```
SELECT CONVERT(DATE,[RateChangeDate]) AS FromDate, CONCAT(LastName,',
',FirstName, ' ', COALESCE(MiddleName,' ')) AS NameInFull, [Rate] * 40 AS SalaryInAWeek
FROM [Person].[Person] AS p
JOIN [HumanResources].[EmployeePayHistory] AS e
ON p.BusinessEntityID = e.BusinessEntityID
WHERE e.RateChangeDate = (
    SELECT MAX(RateChangeDate)
    FROM [HumanResources].[EmployeePayHistory]
    WHERE BusinessEntityID = e.BusinessEntityID
  )
ORDER BY NameInFull ASC;
```

Results Messages

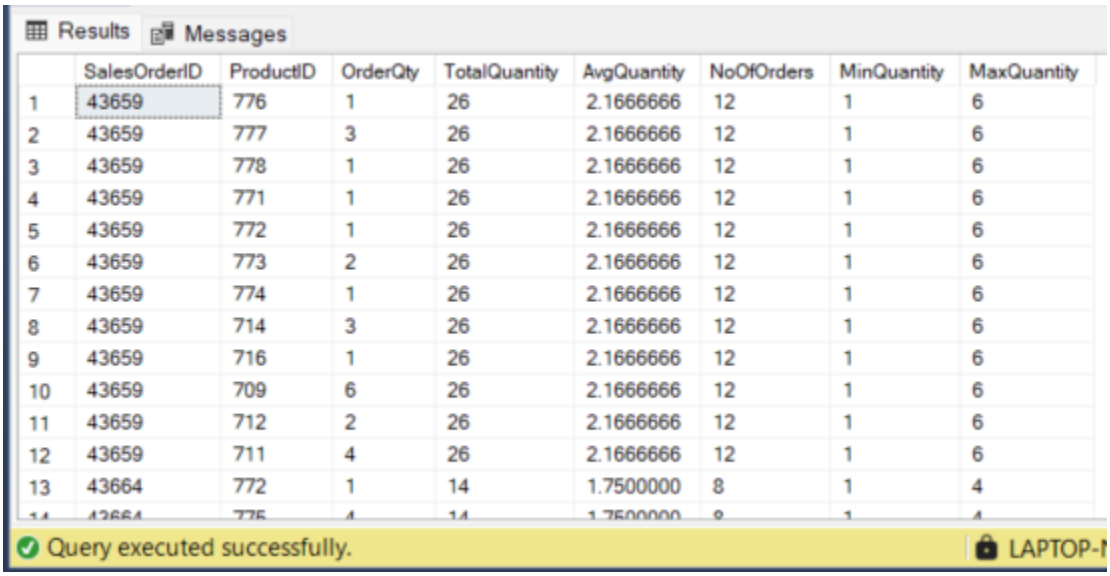
	FromDate	NameInFull	SalaryInAWeek
1	2013-03-14	Abbas, Syed E	1924.04
2	2010-01-16	Abercrombie, Kim B	498.00
3	2009-02-28	Abolrous, Hazem E	1153.848
4	2009-01-02	Ackerman, Pilar G	769.232
5	2009-03-05	Adams, Jay G	498.00
6	2009-01-17	Ajenstat, François P	1538.46
7	2012-04-16	Alberts, Amy E	1924.04
8	2008-12-02	Alderson, Greg F	400.00
9	2008-12-28	Alexander, Sean P	423.076
10	2009-12-02	Altman, Gary E.	961.54
11	2009-01-02	Anderson, Nancy A	498.00
12	2011-05-31	Ansman-Wolfe, Pamela O	923.076
13	2009-01-04	Arifin, Zainal T	711.54
14	2009-01-11	Bacon, Dan K	1096.152

Query executed successfully.

25. Write a query in SQL to find the sum, average, count, minimum, and maximum order quentity for those orders whose id are 43659 and 43664. Return SalesOrderID, ProductID, OrderQty, sum, average, count, max, and min order quantity.

```
SELECT
```

```
    [SalesOrderID], [ProductID], [OrderQty],
  SUM([OrderQty]) OVER (PARTITION BY [SalesOrderID]) AS TotalQuantity,
  AVG(Convert(decimal(18,7),[OrderQty])) OVER (PARTITION BY [SalesOrderID]) AS AvgQuantity,
  COUNT([OrderQty]) OVER (PARTITION BY [SalesOrderID]) AS NoOfOrders,
  MIN([OrderQty]) OVER (PARTITION BY [SalesOrderID]) AS MinQuantity,
  MAX([OrderQty]) OVER (PARTITION BY [SalesOrderID]) AS MaxQuantity
FROM [Sales].[SalesOrderDetail]
WHERE [SalesOrderID] IN (43659, 43664);
```



Results | Messages

	SalesOrderID	ProductID	OrderQty	TotalQuantity	AvgQuantity	NoOfOrders	MinQuantity	MaxQuantity
1	43659	776	1	26	2.1666666	12	1	6
2	43659	777	3	26	2.1666666	12	1	6
3	43659	778	1	26	2.1666666	12	1	6
4	43659	771	1	26	2.1666666	12	1	6
5	43659	772	1	26	2.1666666	12	1	6
6	43659	773	2	26	2.1666666	12	1	6
7	43659	774	1	26	2.1666666	12	1	6
8	43659	714	3	26	2.1666666	12	1	6
9	43659	716	1	26	2.1666666	12	1	6
10	43659	709	6	26	2.1666666	12	1	6
11	43659	712	2	26	2.1666666	12	1	6
12	43659	711	4	26	2.1666666	12	1	6
13	43664	772	1	14	1.7500000	8	1	4

Query executed successfully.